

Software Requirements Specification

Circular Buffer (Ring Buffer) Module for UART Driver

| Field                | Value                                                                                                                        |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|
| Document Identifier  | SRS-RINGBUF-001                                                                                                              |
| Revision             | A                                                                                                                            |
| Date                 | 05 August 2026                                                                                                               |
| Software Level       | DAL C (illustrative)                                                                                                         |
| Applicable Standards | RTCA DO-178C — Software Considerations in Airborne Systems<br>MISRA-C:2012 — Guidelines for the Use of C in Critical Systems |
| Target Language      | C99 (freestanding)                                                                                                           |
| Status               | Released for implementation                                                                                                  |

*This specification is a training artefact produced for portfolio and interview preparation purposes. It is not intended for use in a certified airborne product.*

# 1. Introduction

## 1.1 Purpose

This document specifies the functional and non-functional requirements for a fixed-size circular buffer (ring buffer) software module. The module provides a lock-free single-producer / single-consumer queue intended to decouple a UART receive interrupt service routine (ISR) from the application background loop.

## 1.2 Scope

The module is a reusable, hardware-independent component. It does not configure the UART peripheral, does not manage interrupt priorities, and does not perform any input/output operation. Integration with a specific microcontroller is outside the scope of this specification.

## 1.3 Definitions and Acronyms

| Term            | Definition                                                                                                    |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| <b>Producer</b> | The execution context that writes data into the buffer. In the reference use case, the UART receive ISR.      |
| <b>Consumer</b> | The execution context that reads data from the buffer. In the reference use case, the background (main) loop. |
| <b>SPSC</b>     | Single Producer, Single Consumer.                                                                             |
| <b>HLR</b>      | High-Level Requirement.                                                                                       |
| <b>LLR</b>      | Low-Level Requirement.                                                                                        |
| <b>DAL</b>      | Design Assurance Level.                                                                                       |

## 1.4 Requirement Conventions

The verb **shall** denotes a mandatory requirement subject to verification. Each requirement carries a unique identifier and is traceable to a verification method: **T** (Test), **A** (Analysis), **R** (Review), **I** (Inspection).

## 2. High-Level Requirements

High-level requirements define the externally observable behaviour of the module.

| ID         | Requirement                                                                                                                                                          | Method |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| HLR-RB-001 | The module shall provide a First-In-First-Out (FIFO) circular buffer of unsigned 8-bit elements.                                                                     | T      |
| HLR-RB-002 | The capacity of the buffer shall be fixed at compile time and shall be supplied by the caller as a statically allocated storage area.                                | R      |
| HLR-RB-003 | The module shall not perform any dynamic memory allocation.                                                                                                          | R, A   |
| HLR-RB-004 | The module shall support concurrent access by exactly one producer and one consumer without disabling interrupts and without requiring a mutual-exclusion primitive. | A, T   |
| HLR-RB-005 | Every public function of the module shall execute in constant time, independent of the buffer capacity and of the number of elements stored.                         | A      |
| HLR-RB-006 | The module shall detect and report the buffer-full and buffer-empty conditions to the caller by means of a return status.                                            | T      |
| HLR-RB-007 | The module shall not overwrite unread data when the buffer is full.                                                                                                  | T      |
| HLR-RB-008 | The module shall be free of any dependency on the C standard library, with the exception of the fixed-width integer and size types.                                  | R      |
| HLR-RB-009 | The source code shall comply with MISRA-C:2012 mandatory and required rules. Any deviation shall be documented with a justification.                                 | R, A   |
| HLR-RB-010 | The module shall be reentrant with respect to distinct buffer instances.                                                                                             | R, T   |

## 3. Software Interface

### 3.1 Data Types

| Identifier  | Description                                                                                                                                                                          |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rb_status_t | Enumerated return status. Shall define at least: RB_OK, RB_ERR_PARAM, RB_ERR_FULL, RB_ERR_EMPTY.                                                                                     |
| rb_t        | Opaque control structure holding the storage pointer, the capacity, and the head and tail indices. Shall be declared in the public header to permit static allocation by the caller. |

### 3.2 Public Functions

| Prototype                                                         | Summary                                      |
|-------------------------------------------------------------------|----------------------------------------------|
| rb_status_t rb_init(rb_t *rb, uint8_t *storage, size_t capacity); | Initialise a buffer instance.                |
| rb_status_t rb_put(rb_t *rb, uint8_t byte);                       | Insert one byte. Producer side.              |
| rb_status_t rb_get(rb_t *rb, uint8_t *byte);                      | Remove one byte. Consumer side.              |
| size_t rb_count(const rb_t *rb);                                  | Return the number of bytes currently stored. |
| bool rb_is_empty(const rb_t *rb);                                 | Return the empty condition.                  |
| bool rb_is_full(const rb_t *rb);                                  | Return the full condition.                   |
| rb_status_t rb_flush(rb_t *rb);                                   | Discard all stored data. Consumer side only. |

## 4. Low-Level Requirements

Low-level requirements refine the high-level requirements into a form directly implementable in source code. The **Trace** column identifies the parent requirement.

| ID         | Requirement                                                                                                                                                                                                                                        | Trace      |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| LLR-RB-001 | <b>rb_init</b> shall return RB_ERR_PARAM if rb is NULL, if storage is NULL, if capacity is zero, or if capacity is not an integer power of two.                                                                                                    | HLR-RB-002 |
| LLR-RB-002 | <b>rb_init</b> shall, upon successful validation, store the supplied pointer and capacity, set the head and tail indices to zero, and return RB_OK.                                                                                                | HLR-RB-002 |
| LLR-RB-003 | The buffer shall be considered empty when the head index is equal to the tail index.                                                                                                                                                               | HLR-RB-006 |
| LLR-RB-004 | The buffer shall be considered full when the difference between the head index and the tail index is equal to the capacity.                                                                                                                        | HLR-RB-006 |
| LLR-RB-005 | <b>rb_put</b> shall return RB_ERR_PARAM if rb is NULL.                                                                                                                                                                                             | HLR-RB-006 |
| LLR-RB-006 | <b>rb_put</b> shall return RB_ERR_FULL and shall leave the buffer contents unmodified when the full condition of LLR-RB-004 is satisfied.                                                                                                          | HLR-RB-007 |
| LLR-RB-007 | <b>rb_put</b> shall write the supplied byte to the storage element designated by the head index masked with (capacity minus one), then increment the head index, then return RB_OK. The storage write shall be ordered before the index update.    | HLR-RB-004 |
| LLR-RB-008 | <b>rb_get</b> shall return RB_ERR_PARAM if rb is NULL or if byte is NULL.                                                                                                                                                                          | HLR-RB-006 |
| LLR-RB-009 | <b>rb_get</b> shall return RB_ERR_EMPTY and shall leave the output parameter unmodified when the empty condition of LLR-RB-003 is satisfied.                                                                                                       | HLR-RB-006 |
| LLR-RB-010 | <b>rb_get</b> shall copy the storage element designated by the tail index masked with (capacity minus one) into the output parameter, then increment the tail index, then return RB_OK. The storage read shall be ordered before the index update. | HLR-RB-004 |
| LLR-RB-011 | The head index shall be written by the producer only. The tail index shall be written by the consumer only.                                                                                                                                        | HLR-RB-004 |
| LLR-RB-012 | The head and tail indices shall be of an unsigned integer type whose read and write accesses are atomic on the target architecture, and shall be permitted to wrap around modulo the width of that type.                                           | HLR-RB-004 |
| LLR-RB-013 | <b>rb_count</b> shall return the difference between the head index and the tail index, evaluated in unsigned arithmetic. It shall return zero if rb is NULL.                                                                                       | HLR-RB-005 |
| LLR-RB-014 | <b>rb_flush</b> shall set the tail index equal to the current head index and shall return RB_OK. It shall return RB_ERR_PARAM if rb is NULL.                                                                                                       | HLR-RB-001 |
| LLR-RB-015 | No public function shall contain an unbounded loop, a recursive call, or a call to any other function of the module.                                                                                                                               | HLR-RB-005 |
| LLR-RB-016 | All public functions shall be declared in a single header file protected against multiple inclusion.                                                                                                                                               | HLR-RB-008 |
| LLR-RB-017 | Pointer parameters that are not modified by the function shall be qualified const.                                                                                                                                                                 | HLR-RB-009 |
| LLR-RB-018 | No function shall have more than one return statement (MISRA-C:2012 Rule 15.5).                                                                                                                                                                    | HLR-RB-009 |

## 5. Verification Requirements

The following test cases constitute the minimum acceptance set. Each shall be executed on the host platform. Structural coverage of statements and decisions shall be measured.

| Test ID   | Objective                                                                                                                                        | Covers       |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| TC-RB-001 | Initialisation with a NULL control pointer, a NULL storage pointer, a zero capacity, and a capacity of 12 shall each return RB_ERR_PARAM.        | LLR-001      |
| TC-RB-002 | Initialisation with a valid 16-byte storage area shall return RB_OK and the buffer shall report a count of zero.                                 | LLR-002, 003 |
| TC-RB-003 | A single put followed by a single get shall return the same byte value.                                                                          | LLR-007, 010 |
| TC-RB-004 | Filling the buffer to capacity shall succeed; the subsequent put shall return RB_ERR_FULL and shall not alter the count.                         | LLR-004, 006 |
| TC-RB-005 | A get performed on an empty buffer shall return RB_ERR_EMPTY and shall not modify the output variable.                                           | LLR-009      |
| TC-RB-006 | Writing and reading five thousand bytes through a sixteen-byte buffer shall preserve FIFO order and shall demonstrate correct index wrap-around. | LLR-012      |
| TC-RB-007 | A flush performed on a partially filled buffer shall reduce the count to zero; a subsequent get shall return RB_ERR_EMPTY.                       | LLR-014      |
| TC-RB-008 | Two independent buffer instances shall operate without mutual interference.                                                                      | HLR-010      |

## 6. Deliverables

| Item               | Content                                                             |
|--------------------|---------------------------------------------------------------------|
| ring_buffer.h      | Public interface: types, status codes, function declarations.       |
| ring_buffer.c      | Implementation of all requirements of Section 4.                    |
| test_ring_buffer.c | Test harness implementing Section 5.                                |
| Makefile           | Build and test targets for the host platform.                       |
| README.md          | Build instructions, requirement summary, and traceability overview. |

## 7. Revision History

| Rev | Date        | Description      |
|-----|-------------|------------------|
| A   | 05 Aug 2026 | Initial release. |